1. Single Inheritance – Scenario

Questions:

1. **How would you implement this using single inheritance?**

   Create a Vehicle class and let Car extend it. Override startEngine() in Car to log the engine start time and then call the parent method.

```
class Vehicle {
    void startEngine() {
        System.out.println("Engine Started");
    }
    void stopEngine() {
        System.out.println("Engine Stopped");
    }
}
class Car extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Engine started at: " +
java.time.LocalTime.now());
        super.startEngine();
    }
    void playMusic() {
        System.out.println("Playing Music");
```

```
    }
}
public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.startEngine();
        c.playMusic();
        c.stopEngine();
    }
}
```

2. **Should Car override startEngine()? Why?**

   Yes.
   Because the company wants every car to log the engine start time before starting the engine.

3. **How can the parent class implementation still be executed?**

   Use the super keyword.

4. **What happens if Car does not override startEngine()?**

   - The Vehicle version of startEngine() is executed.
   - Engine start time will **not** be logged.

2. Multilevel Inheritance – Scenario

Questions:

1. **Why is multilevel inheritance suitable here?**

   Because each class adds new features while inheriting existing ones.

   - Employee → Employee ID, Name
   - Developer → Programming Language
   - SeniorDeveloper → Team Size, Code Review

   No code duplication occurs.

2. **Which members are accessible in SeniorDeveloper?**

   SeniorDeveloper can access:

   - Employee ID
   - Employee Name
   - Programming Language
   - Team Size
   - Code Review methods
   - All inherited public/protected methods

3. **If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?**

   SeniorDeveloper inherits the **Developer's overridden method** unless it overrides it again

4. **Draw the inheritance hierarchy.**

Employee

|

Developer

|

SeniorDeveloper

3. Hierarchical Inheritance – Scenario

Questions:

1. **Why is hierarchical inheritance appropriate here?**

   Because multiple payment types share common features but have different processing methods.

   Payment

   / | \

   CreditCard UPI NetBanking

2. **Where should common validation logic be implemented?**

   Inside the **Payment** class.

```
class Payment {
    void validate() {
        System.out.println("Validation Successful");
    }
}
```

3. **How can each payment class provide its own transaction processing?**

   Override the processPayment() method.

   ```
   class CreditCardPayment extends Payment {
      void processPayment() {
         System.out.println("Credit Card Payment");
      }
   }
   ```

4. **What is the advantage over writing three independent classes?**

- Less duplication
- Easier maintenance
- Common functionality written only once

4. Compile-Time Polymorphism (Method Overloading) – Scenario

Questions:

1. **Which OOP concept should be used?**

   Method Overloading (Compile-Time Polymorphism).

   ```
   class Calculator {
      int add(int a, int b) {
         return a + b;
      }
      int add(int a, int b, int c) {
         return a + b + c;
   ```

```
    }
    double add(double a, double b) {
        return a + b;
    }
}
```

2. **How will Java decide which method to invoke?**

   Java checks:

- Number of arguments
- Type of arguments
- Order of arguments

   This decision is made **at compile time**.

3. **Can methods be overloaded by changing only the return type?**

   No.

   add(double, double)

   This causes a compilation error.

4. **What happens if add(5, 10.0) is called?**

   Java selects:

   add(double, double)

   because 5 is automatically converted from int to double.

5. Runtime Polymorphism (Method Overriding) – Scenario

Questions:

1. **Which OOP concept is being used?**

   Runtime Polymorphism (Method Overriding).

2. **Which version of calculateDeliveryTime() executes?**

   The method of the **actual object** executes.

   Restaurant r = new Pizza();

   r.calculateDeliveryTime();

   Output:

   Pizza delivery time

3. **When is the method selection decided?**

   At **runtime**.

4. **What would happen if calculateDeliveryTime() were declared static?**

   Static methods are **not overridden**.

   The method called depends on the **reference type**, not the object type.

   So runtime polymorphism will **not work**.

6. Interface – Scenario

Questions:

1. **Would you use an interface or inheritance? Why?**

   Use an **Interface** because:

- Different devices have different implementations.
- Third-party companies can easily create new devices.
- It supports loose coupling.

```
interface SmartDevice {
    void turnOn();
    void turnOff();
}
class SmartBulb implements SmartDevice {
    public void turnOn() {
        System.out.println("Bulb ON");
    }
    public void turnOff() {
        System.out.println("Bulb OFF");
    }
}
```

2. **Design an appropriate interface.**

```
interface SmartDevice {
    void turnOn();
    void turnOff();
}
class SmartBulb implements SmartDevice {
    public void turnOn() {
```

```
        System.out.println("Bulb ON");
    }
    public void turnOff() {
        System.out.println("Bulb OFF");
    }
}
```

3. **Can one device implement multiple interfaces? Give an example.**

Yes

```
interface SmartDevice {
    void turnOn();
    void turnOff();
}
interface WiFiEnabled {
    void connectWiFi();
}
class SmartAC implements SmartDevice, WiFiEnabled {
    public void turnOn() {
        System.out.println("AC ON");
    }
    public void turnOff() {
        System.out.println("AC OFF");
```

```
    }

    public void connectWiFi() {

        System.out.println("WiFi Connected");

    }

}
```

4. **Why is an interface better than creating a common base class in this scenario?**

- Supports multiple interface implementation.
- Allows third-party manufacturers to add devices easily.
- Promotes loose coupling.
- Different devices can have completely different implementations while following the same contract.